

Entendendo o C4 Model: Uma Abordagem para Arquitetura de Software



Allan Santos · Follow

Published in cajudevs · 10 min read · Feb 23, 2024



12



2



Este artigo tem como objetivo realizar a fixação de conhecimento acerca do C4 Model e contribuir com a comunidade com um levantamento sobre suas características, conceitos e exemplos.

Caso encontre qualquer erro ou deseje realizar algum feedback, pode ficar à vontade que realize as alterações necessárias. O foco principal é ter mais uma fonte de pesquisa e estudo para quem se interessar pelo C4 Model.

Introdução

Nos ambientes de desenvolvimento de software complexos e em constante evolução, a compreensão e comunicação eficazes da arquitetura são essenciais para o sucesso de um projeto. É aqui que o C4 Model se destaca, oferecendo uma abordagem visual e intuitiva para representar e comunicar arquiteturas de software. Neste artigo, vamos explorar o que é o C4 Model, seus níveis de abstração e por que é uma escolha valiosa para arquitetos e desenvolvedores.

Por que Documentar Software?

A documentação de software desempenha um papel crucial no desenvolvimento de sistemas de software de alta qualidade. Ela fornece um registro permanente das decisões de design, arquitetura e implementação, facilitando a compreensão, manutenção e evolução do software ao longo do tempo. Essa prática é particularmente relevante no contexto do Manifesto Ágil, que valoriza indivíduos e interações sobre processos e ferramentas, mas também reconhece a importância da documentação abrangente como uma ferramenta de comunicação e colaboração eficaz.

Como e Quando Surgiu?

O C4 Model foi introduzido por Simon Brown, um experiente arquiteto de software, como parte de seu livro “Software Architecture for Developers”. A necessidade de uma abordagem mais clara e consistente para comunicar arquiteturas de software tornou-se evidente à medida que os sistemas se tornaram mais complexos e distribuídos. O C4 Model surgiu como uma resposta a essa necessidade, oferecendo uma estrutura simples para representar arquiteturas de software em diferentes níveis de detalhe.

Mas o que é o C4 Model?

O C4 Model é uma abordagem para modelar arquiteturas de software, organizada em quatro níveis de abstração: Contexto, Contêiner, Componente e Código. Cada nível oferece uma visão única da arquitetura, desde uma visão geral de alto nível até os detalhes de implementação. A nomenclatura representa justamente uma abreviação de cada nível: *Context*, *Container*, *Component* e *Code*.

O C4 Model caracteriza-se por ser uma notação gráfica limpa de arquitetura de sistemas. O modelo consegue exibir vários níveis de detalhes e fornece uma maneira para que os desenvolvedores de software e partes interessadas tenham compreensão do sistema sem se ater muito aos detalhes. Isso é um ponto interessante, pois resolve parcialmente um dos grandes problemas que existem entre gestores de negócio e a área técnica, que é a comunicação.

Dessa forma, o C4 Model basicamente consegue entregar uma comunicação mais clara, documentação facilitada e colaboração na manutenção e evolução de software.

Alguns Casos de Uso para o C4 Model

Qualidade e Testes

O C4 Model pode ser útil para testes e garantia de qualidade provendo claro entendimento da arquitetura do sistema e permitindo que os Analistas de Qualidade criem estratégias de testes de fácil compreensão. Além disso, ajuda a garantir que todos os componentes sejam testados de forma satisfatória.

O nível de Contexto e Contêiner provê o conhecimento da interação com sistemas externos com testes mais robustos e o de Componente admite a criação de testes mais detalhados assegurando assim, que as funcionalidades operam como esperado.

Arquitetura de Microsserviços

A abordagem muito utilizada no mercado, o C4 Model pode ajudar bastante na compreensão e documentação desse tipo de arquitetura.

O nível de Contexto fornece uma visão de alto nível com as interações entre os serviços internos e externos. Da ótica do Container possui mais detalhes fornecendo como cada microsserviço é composto, podendo conter banco de dados, servidores web, APIs, aplicativos mobile ou qualquer outra infraestrutura. O nível de Componente se aprofunda ainda mais exibindo detalhes internos de cada microsserviço como por exemplo Interfaces e Controllers. Para fechar, o nível de Código proporciona assinatura de métodos, trechos de códigos e mais detalhes de implementação.

Auditorias de Segurança

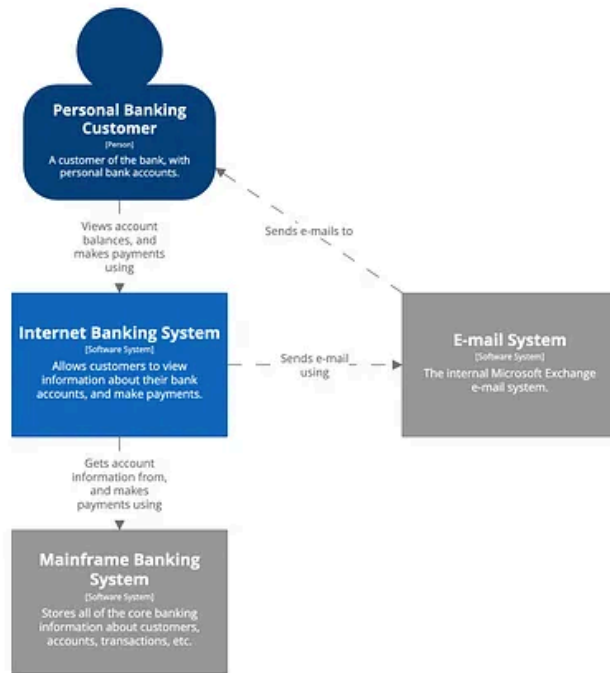
O C4 Model pode auxiliar times de Segurança no entendimento da arquitetura do sistema com o objetivo de identificar possíveis vulnerabilidades e assim mitigá-las.

No alto nível, o Contexto exibe como as interações acontecem entre as partes permitindo possíveis pontos inseguros de comunicação interna ou externa. As visões de Contêiner e Componente oferecem um pouco mais de detalhes onde fornece dados são armazenados, por exemplo. Dessa forma podem ser identificados bancos de dados sem a devida proteção e surgir ideias para novas medidas de segurança. Por fim, o nível de Código propicia revisão de códigos com objetivo de apontar falhas de segurança.

Os Níveis do C4 Model

Nível 1: Contexto

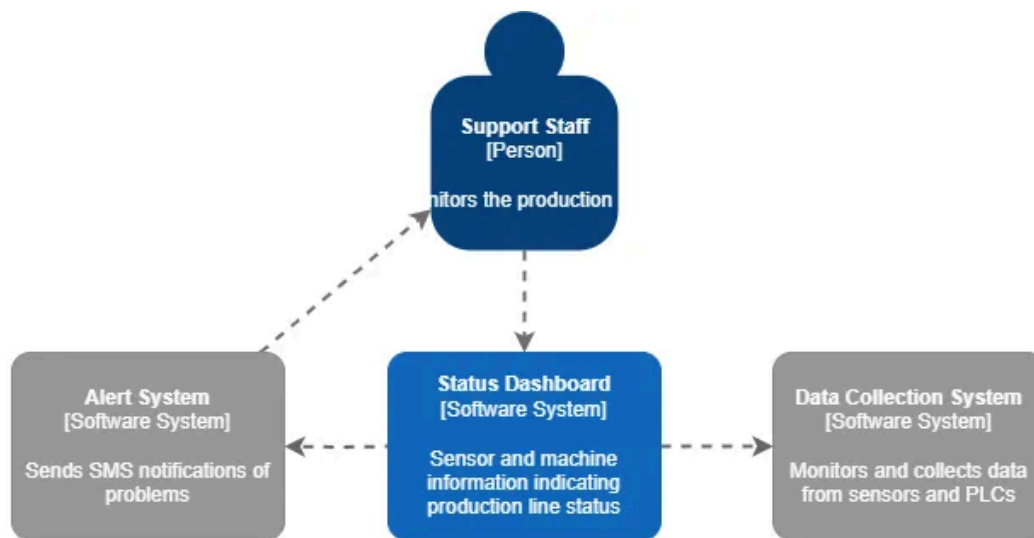
O nível de contexto fornece uma visão panorâmica do sistema, identificando os principais elementos e suas interações de alto nível. Isso inclui usuários, sistemas externos e os principais componentes do sistema em questão. Um diagrama de contexto do C4 Model é uma ferramenta poderosa para alinhar as partes interessadas sobre os objetivos e escopo do sistema.



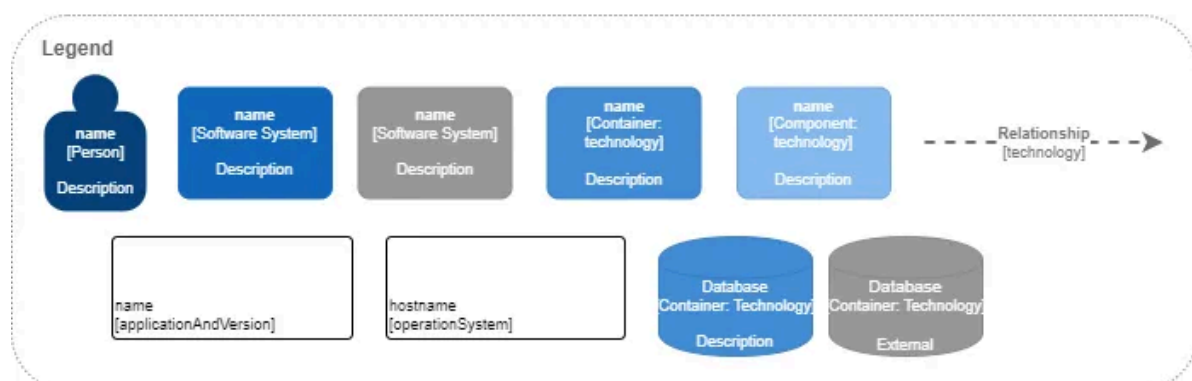
[System Context] Internet Banking System

The system context diagram for the Internet Banking System - diagram created with Structurizr.
Saturday, 11 November 2023 at 09:04 Greenwich Mean Time

Exemplo 1: Diagrama de nível 1 em C4, disponível em <https://c4model.com/>



Status Dashboard (Context)

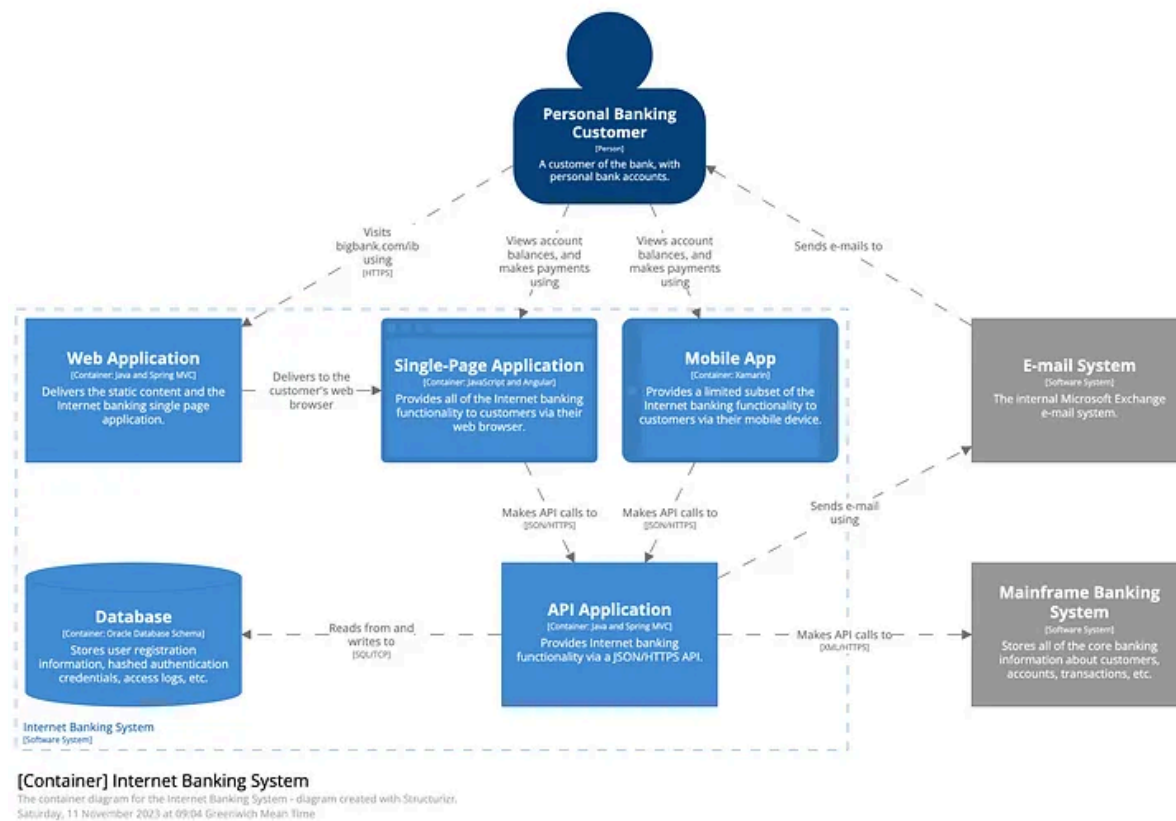


Exemplo 2: Diagrama de nível 1 em C4, disponível em [Draw.io](https://draw.io) Examples C4 Model

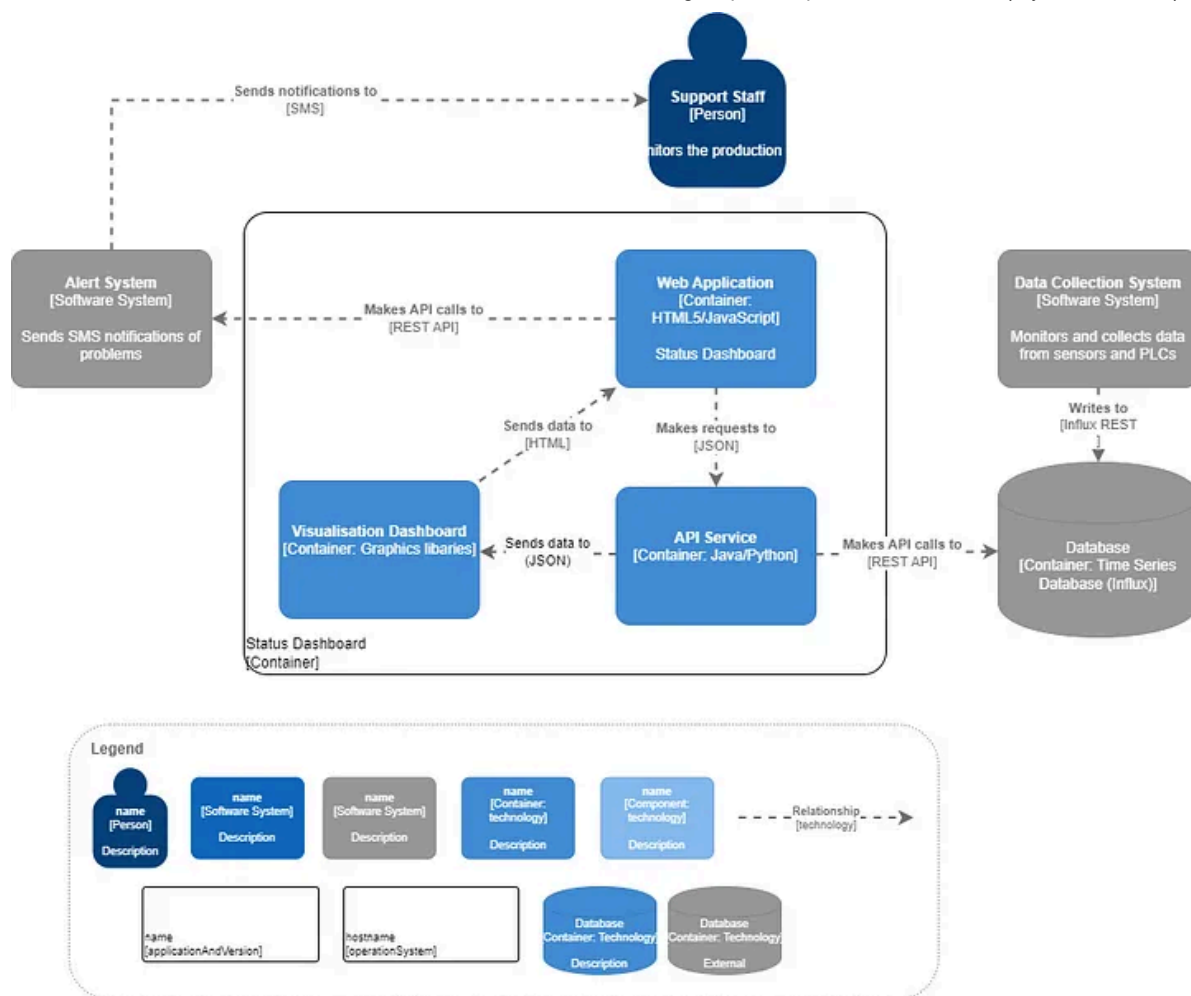
Nível 2: Contêiner

No nível de contêiner, detalhamos os sistemas ou subsistemas de software em termos de contêineres, como aplicativos, bancos de dados, serviços, entre outros, e suas interações. Isso ajuda a entender a estrutura de alto nível e as dependências entre os componentes. Diagramas de contêineres são úteis para identificar os principais subsistemas de um sistema e como eles se comunicam entre si.

É essencial destacar que o nível de Contêiner no C4 Model não está vinculado ao Docker especificamente. Dentro deste modelo, refere-se à capacidade de executar código ou armazenar dados, como um aplicativo móvel, um banco de dados, armazenamento de arquivos, um servidor backend, e assim por diante.

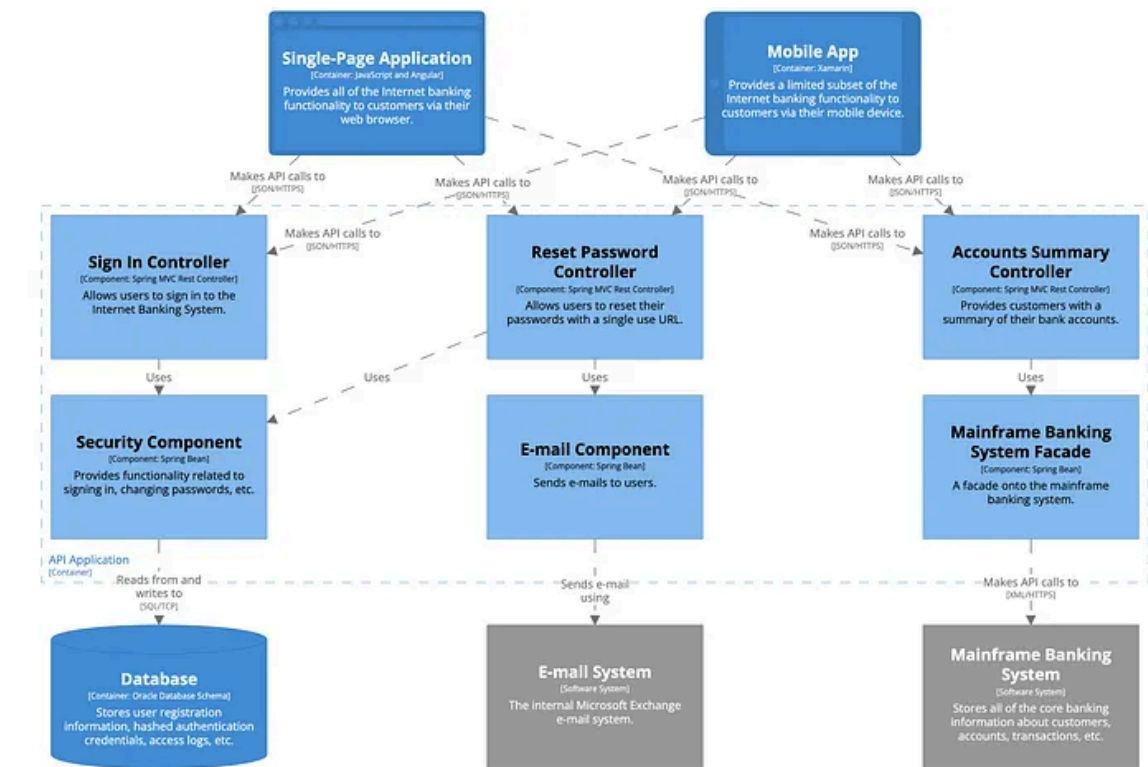


Exemplo 3: Diagrama de nível 2 em C4, disponível em <https://c4model.com/>

Exemplo 4: Diagrama de nível 2 em C4, disponível em [Draw.io Examples C4 Model](#)

Nível 3: Componente

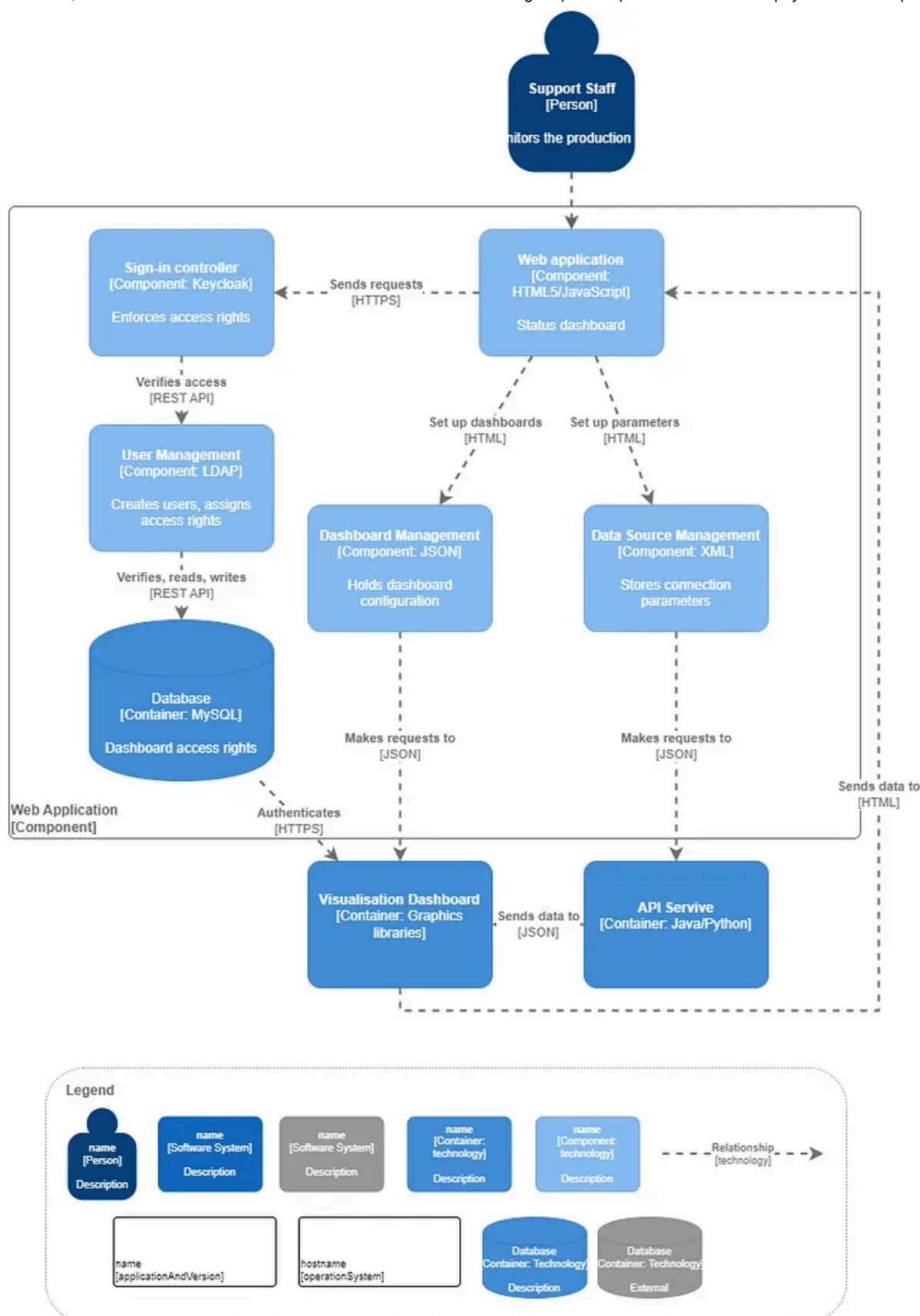
No nível de componente, exploramos os detalhes internos de um contêiner, identificando os componentes individuais e suas interações. Isso ajuda a entender como os diferentes módulos ou serviços colaboram para realizar funcionalidades específicas. Diagramas de componentes são valiosos para entender a estrutura interna de um subsistema e as dependências entre os diferentes componentes.



[Component] Internet Banking System - API Application

The component diagram for the API Application - diagram created with Structurizr.
Saturday, 11 November 2023 at 09:04 Greenwich Mean Time

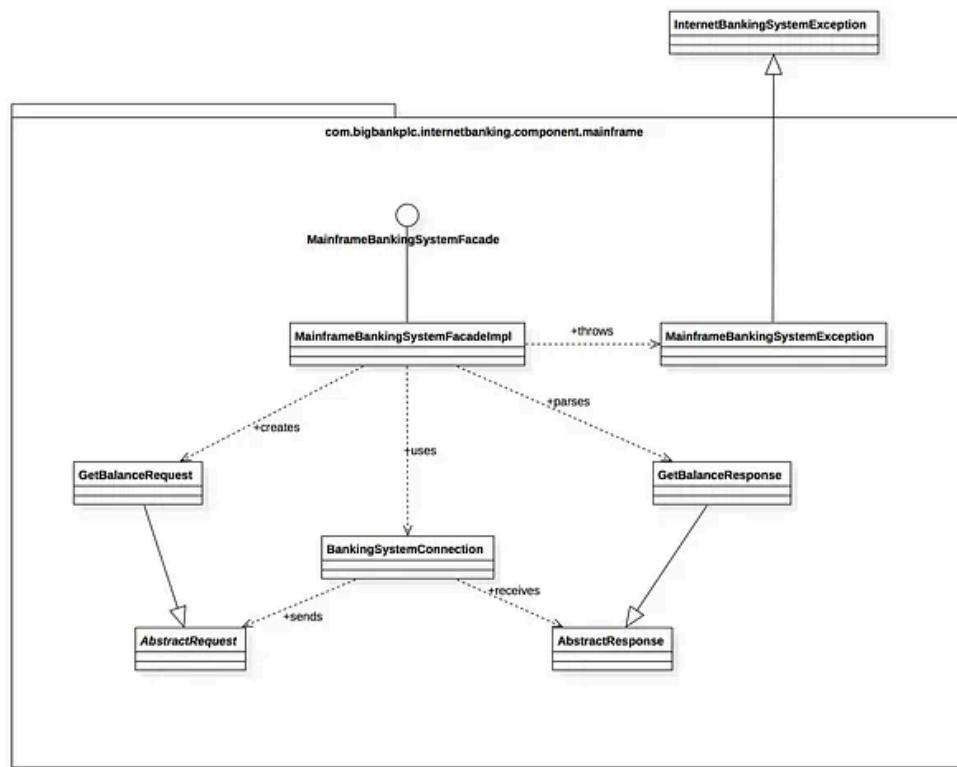
Exemplo 5: Diagrama de nível 3 em C4, disponível em <https://c4model.com/>

Exemplo 6: Diagrama de nível 3 em C4, disponível em [Draw.io Examples C4 Model](https://draw.io/Examples/C4%20Model/)

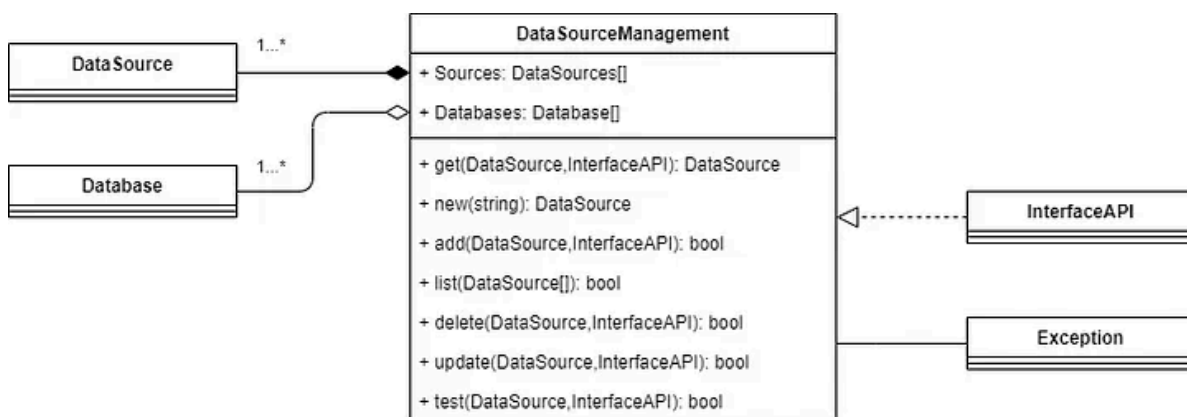
Nível 4: Código

No nível de código, entramos nos detalhes de implementação dos componentes, incluindo diagramas de classes, diagramas de sequência, entre outros artefatos que mostram como o

código é estruturado e como as classes e métodos se relacionam. Esses diagramas mostram como o código é organizado e como as diferentes partes se conectam. Embora não seja obrigatório, esse nível oferece uma visão bem detalhada de como o sistema é implementado, o que é especialmente útil para os desenvolvedores que precisam entender e realizar manutenções no código-fonte.



Exemplo 7: Diagrama de nível 4 em C4, disponível em <https://c4model.com/>



Data Source Management
(UML Class)

Exemplo 8: Diagrama de nível 4 em C4, disponível em [Draw.io Examples C4 Model](https://draw.io/Examples/C4/Model/)

Ferramentas para criar C4 Model

Draw.io

O [Draw.io](https://draw.io) é uma ferramenta que serve para construir diagramas baseado em navegador mais utilizada do mundo. Com um acervo muito grande de bibliotecas, o C4 Model também está presente nesta ferramenta. Acesse o [site](https://draw.io) para criar seu próprio diagrama e habilite no painel esquerdo (selecionar a opção “Mais Formas”) a biblioteca do C4 Model e depois clique em aplicar.



Opção para adicionar a biblioteca de C4 Model no draw.io.

Dessa forma você poderá criar diagramas utilizando a biblioteca com as formas do C4 Model.

Structurizr

O [Structurizr](https://structurizr.com) baseia-se em “Diagram as a Code” para construir diversos diagramas de arquitetura de softwares a partir de um único modelo. Existem ferramentas para trabalhar com o Structurizr e a mais recomendada para a maioria das equipes é a [Structurizr DSL](https://structurizr.com). A Structurizr DSL fornece um modo de arquitetura baseado no C4 Model que utiliza uma linguagem de domínio (DSL — Domain Specific Language) que é baseada em texto.

A documentação completa pode ser encontrada neste [link](#) e explica como cada comando pode ser utilizada. Alguns exemplos de comandos utilizados nos exemplos serão exibidas abaixo e suas devidas explicações:

- ***workspace***: corresponde à linguagem de nível mais alto do código e contém os propriedades, *views*, *model*, entre outros. Pode opcionalmente ter um nome e descrição.

```
workspace [name] [description] {  
    ...  
}
```

- ***model***: cada *workspace* contém um *model* onde elementos e relacionamentos são definidos.

```
model {  
    ...  
}
```

- ***group***: provê um modo de definir um grupo nomeado de elementos, que serão renderizados com sua “fronteira” diante dos outros elementos.

```
group <name> {  
    ...  
}
```

- ***person***: define um tipo de “pessoa” como por exemplo: usuário, ator, regra, etc).

```
person <name> [description] [tags] {  
    ...  
}
```

- ***softwareSystem***: como o próprio nome já diz, representa um *software*.

```
softwareSystem <name> [description] [tags] {
    ...
}
```

- ***container***: define o tipo Contêiner do C4 Model e, assim como outros elementos, podem ter nome, descrição, tecnologia e *tags*.

```
container <name> [description] [technology] [tags] {
    ...
}
```

Exemplo de criação com código para o “Exemplo 1: Diagrama de nível 1 (Context) em C4”:

```
workspace "Internet Banking System" {

    model {
        user = person "Personal Banking Customer" "A customer of the bank with personal bank acc

        internetBankingSystem = softwareSystem "Internet Banking System." "Allows customers to v

        emailSystem = softwareSystem "E-mail system" "The bank's Microsoft Exchange system." "Ex
        mainframe = softwareSystem "Mainframe Banking System" "Stores all of the corebanking inf

        user -> internetBankingSystem "Views account balances and makes payments using"

        internetBankingSystem -> emailSystem "Sends e-mail using " "" "Dashed"
        emailSystem -> user "Sends emails to" "E-mail message" "Dashed"
        internetBankingSystem -> mainframe "Gets account information from, and makes payments us

    }

    views {

        systemContext internetBankingSystem "Context" "" {
            include *
            autoLayout
        }
    }
}
```

```

styles {
  element "Software System" {
    background #1168bd
    color #ffffff
  }
  element "Person" {
    shape person
    background #08427b
    color #ffffff
  }
  element "External" {
    background #999999
    color #ffffff
  }
  relationship "Relationship" {
    dashed false
  }
  relationship "Dashed" {
    dashed true
  }
}

```

Exemplo de criação com código para o “Exemplo 1: Diagrama de nível 2 (Container) em C4”:

```

workspace "Big Bank plc" "This is an example workspace to illustrate the key features of Structurizr"

model {
  customer = person "Personal Banking Customer" "A customer of the bank, with personal bank account"

  group "" {

    mainframe = softwareSystem "Mainframe Banking System" "Stores all of the core banking data"
    email = softwareSystem "E-mail System" "The internal Microsoft Exchange e-mail system"

    internetBankingSystem = softwareSystem "Internet Banking System" "Allows customers to access banking services online"
    singlePageApplication = container "Single-Page Application" "Provides all of the user interface for the internet banking system"
    mobileApp = container "Mobile App" "Provides a limited subset of the Internet banking services"
    webApplication = container "Web Application" "Delivers the static content and the dynamic content for the internet banking system"
    apiApplication = container "API Application" "Provides Internet banking functionality"
    database = container "Database" "Stores user registration information, hashed authentication credentials, and other data"

  }

}

# relationships between people and software systems
customer -> internetBankingSystem "Views account balances, and makes payments using"
internetBankingSystem -> mainframe "Gets account information from, and makes payments using"
internetBankingSystem -> email "Sends e-mail using"
email -> customer "Sends e-mails to"

```

```

# relationships to/from containers
customer -> webApplication "Visits bigbank.com/ib using" "HTTPS"
customer -> singlePageApplication "Views account balances, and makes payments using"
customer -> mobileApp "Views account balances, and makes payments using"
webApplication -> singlePageApplication "Delivers to the customer's web browser"
apiApplication -> email "Sends e-mail using"
singlePageApplication -> apiApplication "Makes API calls to" "JSON/HTTPS"
mobileApp -> apiApplication "Makes API calls to" "JSON/HTTPS"
apiApplication -> database "Reads from and writes to" "SQL/TCP"
apiApplication -> mainframe "Makes API calls to" "XML/HTTPS"
}

views {

  container internetBankingSystem "Containers" {
    include *
    autoLayout
    description "The container diagram for the Internet Banking System."
  }

  styles {
    element "Person" {
      color #ffffff
      fontSize 22
      shape Person
    }
    element "Customer" {
      background #08427b
    }
    element "Bank Staff" {
      background #999999
    }
    element "Software System" {
      background #1168bd
      color #ffffff
    }
    element "Existing System" {
      background #999999
      color #ffffff
    }
    element "Container" {
      background #438dd5
      color #ffffff
    }
    element "Web Browser" {
      shape WebBrowser
    }
    element "Mobile App" {
      shape MobileDeviceLandscape
    }
    element "Database" {
      shape Cylinder
    }
    element "Component" {
      background #85bbf0
      color #000000
    }
    element "Failover" {

```

```
opacity 25
```

Por que então adotar o C4 Model para Arquitetura dos Projetos?

Haja vista conceitos e características, pode-se afirmar algumas razões para adotar o C4 Model na arquitetura de projetos de software:

- **Comunicação Efetiva:** O C4 Model fornece uma linguagem simples e comum para descrever arquiteturas de software, facilitando a comunicação entre diferentes partes interessadas, como desenvolvedores, arquitetos, gestores e clientes.
- **Clareza e Concisão:** A abordagem visual do C4 Model torna mais fácil entender a arquitetura do sistema em diferentes níveis de detalhe, desde uma visão geral até os detalhes de implementação.
- **Flexibilidade e Escalabilidade:** O C4 Model é flexível o suficiente para se adaptar a uma variedade de contextos e projetos, desde pequenas aplicações até sistemas distribuídos complexos.
- **Facilidade de Manutenção:** A documentação gerada usando o C4 Model é fácil de manter e atualizar à medida que o sistema evolui, garantindo que a arquitetura permaneça precisa e atualizada.

O C4 Model proporciona uma abordagem prática e eficaz para modelagem de arquiteturas de software. Ele promove uma comunicação clara, uma compreensão compartilhada e simplifica a manutenção dos sistemas de software. Ao adotar o C4 Model, as equipes podem colaborar de maneira mais eficiente, tomar decisões embasadas e desenvolver sistemas robustos e escaláveis.

Referências

- <https://c4model.com/>
- <https://medium.com/news-uk-technology/c4-model-a-better-way-to-visualise-software-architecture-df41e5ac57b8>

- <https://sheldonrcohen.medium.com/understanding-the-c4-model-for-software-architecture-documentation-e59c4edd0d56>
- <https://xantycg.medium.com/c4model-architecture-diagrams-for-everyone-8b4d5fb88686>
- <https://medium.com/pravaler-digital-team/c4-model-9b6e56705496>
- https://www.drawio.com/blog/c4-modelling?utm_source=c4modeltools&utm_medium=listing&utm_campaign=c4modeltools_listing
- <https://engineering.linecorp.com/en/blog/diagramming-software-architecture-using-c4-model-and-c4-plantuml>
- <https://www.codesee.io/learning-center/c4-model>
- <https://structurizr.com/dsl>
- <https://github.com/structurizr/examples/blob/main/dsl/big-bank-plc/workspace.dsl>